

Derivação Sintática de um Comando if em Java

1. Linguagem escolhida e fonte da gramática

A linguagem de programação escolhida para esta atividade é Java. A fonte utilizada para consulta da gramática é a Java Language Specification (JLS), documentação oficial da linguagem Java disponibilizada pela Oracle.

Fonte: Java Language Specification — Oracle

<https://docs.oracle.com/en/java/javase/26/docs/specs/jls/index.html>

A especificação da linguagem apresenta as regras de sintaxe utilizadas para definir a estrutura dos programas Java. Para esta atividade, foi utilizado um subconjunto simplificado da gramática, contendo somente as regras necessárias para representar uma estrutura condicional if. A notação utilizada pela especificação do Java é baseada em uma forma de BNF (Backus-Naur Form), utilizada para representar as regras sintáticas da linguagem.

2. Código a ser gerado

O trecho de código escolhido para a derivação representa uma estrutura condicional que verifica se uma pessoa é maior de idade:

```
if (idade >= 18) {  
    System.out.println("Maior de idade");  
}
```

O programa verifica o valor armazenado na variável idade. Caso esse valor seja maior ou igual a 18, a mensagem "Maior de idade" será exibida. O trecho foi escolhido por apresentar uma estrutura relativamente simples, permitindo demonstrar claramente a aplicação das regras de produção da gramática.

3. Regras gramaticais utilizadas

Para facilitar a demonstração, foi selecionado um subconjunto da gramática do Java. As regras abaixo são uma simplificação das estruturas sintáticas correspondentes da Java Language Specification. Os símbolos escritos entre < > representam símbolos não terminais. Os demais elementos representam símbolos terminais.

Regras utilizadas

<if_statement> ::= if (<expression>) <statement>

<statement> ::= <block>

<block> ::= { <statement_list> }

<statement_list> ::= <statement_content>

<statement_content> ::= <expression_statement>

<expression_statement> ::= <method_invocation> ;

<expression> ::= <identifier> <relational_operator> <integer_literal>

<relational_operator> ::= >=

<identifier> ::= idade

<integer_literal> ::= 18

<method_invocation> ::= System.out.println (<string_literal>)

<string_literal> ::= "Maior de idade"

Significado das principais regras

<if_statement> representa uma estrutura condicional if. Ela é formada pela palavra reservada if, uma expressão entre parênteses e uma sentença.

<expression> representa a condição que será avaliada. Neste exemplo, ela é formada pelo identificador idade, pelo operador >= e pelo número 18.

<statement> representa uma sentença da linguagem. Neste caso, ela será substituída por um bloco de código.

<block> representa um bloco delimitado pelas chaves { e }.

<expression_statement> representa uma sentença formada por uma chamada de método seguida de ponto e vírgula.

<method_invocation> representa a chamada do método System.out.println, utilizada para exibir uma mensagem na tela.

<string_literal> representa uma sequência de caracteres, neste caso "Maior de idade".

4. Derivação sintática

A derivação começa pelo símbolo não terminal <if_statement>. A partir dele, são aplicadas sucessivamente as regras de produção até que todos os símbolos não terminais sejam substituídos por símbolos terminais.

Passo 1 - Símbolo inicial

<if_statement>

Aplicando a regra:

<if_statement> ::= if (<expression>) <statement>

obtemos:

<if_statement>
if (<expression>) <statement>

Passo 2 - Derivação da expressão

A regra utilizada para <expression> é:

<expression> ::= <identifier> <relational_operator> <integer_literal>

Então:

if (<expression>) <statement>

if (<identifier> <relational_operator> <integer_literal>) <statement>

Substituindo <identifier> por idade:

if (idade <relational_operator> <integer_literal>) <statement>

Substituindo <relational_operator> pelo operador >=:

if (idade >= <integer_literal>) <statement>

Substituindo <integer_literal> pelo valor 18:

if (idade >= 18) <statement>

Passo 3 - Derivação do bloco

A regra utilizada para <statement> é:

<statement> ::= <block>

Portanto:

if (idade >= 18) <statement>

⇒ if (idade >= 18) <block>

Agora aplicamos:

<block> ::= { <statement_list> }

Obtendo:

if (idade >= 18) { <statement_list> }

A lista de comandos será representada pela regra:

<statement_list> ::= <statement_content>

Assim:

if (idade >= 18) { <statement_content> }

Aplicando a regra:

<statement_content> ::= <expression_statement>

temos:

```
if ( idade >= 18 ) { <expression_statement> }
```

Passo 4 - Derivação da chamada de método

Aplicando:

```
<expression_statement> ::= <method_invocation> ;
```

obtemos:

```
if ( idade >= 18 ) { <method_invocation> ; }
```

Agora aplicamos a regra:

```
<method_invocation> ::= System.out.println ( <string_literal> )
```

Resultado:

```
if ( idade >= 18 ) { System.out.println ( <string_literal> ) ; }
```

Por fim, aplicamos:

```
<string_literal> ::= "Maior de idade"
```

Obtendo:

```
if ( idade >= 18 ) { System.out.println ( "Maior de idade" ) ; }
```

Derivação completa

De forma resumida, toda a derivação pode ser apresentada da seguinte maneira:

```
if_statement>  
  if ( <expression> ) <statement>  
  if ( <identifier> <relational_operator> <integer_literal> ) <statement>  
if ( idade <relational_operator> <integer_literal> ) <statement>  
if ( idade >= <integer_literal> ) <statement>  
if ( idade >= 18 ) <statement>  
if ( idade >= 18 ) <block>  
if ( idade >= 18 ) { <statement_list> }  
if ( idade >= 18 ) { <statement_content> }  
if ( idade >= 18 ) { <expression_statement> }  
if ( idade >= 18 ) { <method_invocation> ; }  
if ( idade >= 18 ) { System.out.println ( <string_literal> ) ; }  
if ( idade >= 18 ) { System.out.println ( "Maior de idade" ) ; }
```

5. Resultado final

Após a substituição de todos os símbolos não terminais, chegamos ao seguinte código Java:

```
if (idade >= 18) {  
    System.out.println("Maior de idade");  
}
```

Esse código representa uma estrutura condicional válida em Java. A condição `idade >= 18` verifica se o valor da variável `idade` é maior ou igual a 18. Caso a condição seja verdadeira, o comando `System.out.println()` será executado e exibirá a mensagem "Maior de idade".

6. Identificação dos símbolos terminais e não terminais

Na gramática utilizada, os símbolos não terminais são aqueles que aparecem entre `< >`. Eles representam elementos sintáticos que ainda precisam ser substituídos por outras regras.

Símbolos não terminais

```
<if_statement>
<statement>
<block>
<statement_list>
<statement_content>
<expression_statement>
<expression>
<relational_operator>
<identifier>
<integer_literal>
<method_invocation>
<string_literal>
```

Os símbolos terminais são os elementos que aparecem no código final e não precisam mais ser substituídos.

Símbolos terminais

```
if
(
)
{
}
idade
>=
18
System.out.println
;
"Maior de idade"
```

Por exemplo, na regra:

```
<expression> ::= <identifier> <relational_operator> <integer_literal>
```

os três elementos entre `< >` são não terminais. Durante a derivação, eles são substituídos por:

```
idade >= 18
```

Nesse resultado, `idade`, `>=` e `18` são símbolos terminais.

7. Explicação final

A derivação sintática demonstra como um trecho de código Java pode ser construído a partir de regras gramaticais. O processo começa com o símbolo não terminal `<if_statement>`, que representa uma estrutura condicional.

A partir desse símbolo, são aplicadas diferentes regras de produção. Primeiro, é construída a estrutura básica do `if`, formada pela palavra reservada `if`, uma condição entre parênteses e um bloco de comandos. Depois, a condição é formada utilizando um identificador, um operador relacional e um número inteiro.

Em seguida, o comando que fica dentro do bloco é construído por meio da chamada do método `System.out.println`, que recebe uma cadeia de caracteres como argumento.

Ao final da derivação, todos os símbolos não terminais foram substituídos por símbolos terminais, resultando no código:

```
if (idade >= 18) {  
    System.out.println("Maior de idade");  
}
```

Portanto, a derivação mostra que o código é sintaticamente válido porque pode ser obtido por meio da aplicação sucessiva das regras da gramática selecionada. Dessa forma, é possível relacionar os conceitos de gramática formal, símbolos terminais, símbolos não terminais, produções e derivação com uma linguagem de programação real.